Student ID:1113715 Student Name:曾喜杰

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Due date: 2025.12.30 23:59:59

Important Notice – Use of AI Tools

In this assignment, you must use at least one AI assistant (e.g. ChatGPT, Gemini, Claude, Grok, M365 Copilot) as a learning tool to help you:

- review definitions,
- compare tree variants, and
- organize your report.

You are not allowed to let the AI directly produce your final diagrams or final report content without your own understanding and rewriting.

You must log all AI prompts and services used (see "AI Usage Log" section below).

1. Goal of This Assignment

In the lectures, we introduced the concept of the tree as a data structure, starting from the general tree and then moving to more specialized forms.

In this assignment, you will:

a. Understand and clearly define:
   1. General tree
   2. Binary tree
   3. Complete binary tree
   4. Binary search tree (BST)
   5. AVL tree
   6. Red-Black tree
   7. Max heap
   8. Min heap

b. Build a hierarchy and transformation path from the general tree to these variants, and explain how each variant adds more structure or constraints.

c. Use a fixed list of integers to construct multiple tree variants and visualize them.

d. Choose one real-world application for each tree type and explain why that data structure fits the application.

e. Practice using AI tools as study companions and keep a simple Q&A log.

2. Given Data

Use the following 20 integers as the input data for all your tree constructions:

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

You will reuse this same sequence for every tree type (binary tree, complete binary tree, BST, AVL, Red-Black, max heap, min heap).

3. Deliverables

Please complete your work in the Student Worksheet Companion and upload it to the YZU Portal System.

Your report should include the following parts:

a. Definitions (Concept Review)

Provide clear, concise definitions for each of the following:

1. General tree
2. Binary tree
3. Complete binary tree
4. Binary search tree (BST)
5. AVL tree
6. Red-Black tree
7. Max heap
8. Min heap

You are encouraged to use AI tools to help you understand these concepts, but you must rewrite the definitions in your own words.

b. Hierarchy and Transformation of Tree Variants

Based on the definitions above, build a “tree family hierarchy” that shows how these structures are related. For example:

- General tree → Binary tree
- Binary tree → Complete binary tree / Binary search tree
- BST → AVL tree / Red-Black tree
- Binary tree → Max heap / Min heap

Tasks:

- Draw a diagram or flow chart that shows the transformation or specialization path: general tree → binary tree → complete binary tree → BST → AVL/Red-Black, etc.
- For each arrow (transformation), briefly explain:
  - What new constraint or property is added?
  - e.g., “Binary tree = tree with at most 2 children per node”,
    “BST = binary tree with left < root < right”,
    “AVL = BST with strict height-balance rule”, etc.

c. Tree Construction with the Given Integers

Using the given 20 integers, construct the following tree variants:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)

4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Important Hint / Restriction:

- For these trees, you must use tree visualization tools (e.g., online visualizers or software) to build and display the tree.
- You may not ask AI tools to directly generate the final tree pictures for you.
- Instead:
  - Use AI only to help you understand algorithms,
  - Then apply those algorithms in a visualizer (or your own implementation).

What to submit for this part:

For each tree type:

- A snapshot (image) of the constructed tree.
- The URL / name of the visualization tool you used.
- A short note on how you inserted the integers (e.g., “insert in the given order as BST”, “build max heap using heapify”, etc.).

d. Application Example for Each Tree

For each of the following:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)
4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Choose one application (real-world or system-level) and explain:

1. Application description
   - e.g., priority scheduling, dictionary lookup, memory allocation, database indexing, etc.
2. Why this tree structure fits
   - What property of this data structure makes it suitable?
   - Example:
     - Max heap → good for priority queue because the largest element is always at the root, so extracting max is efficient.
     - Red-Black tree → good for standard library maps/sets because it guarantees O(log n) operations even under many insertions/deletions.

Your explanation should show that you understand the link between the data structure and its use case.

e. Report Layout and Organization

You are free to design the layout of your report, but it should:

- Be well-structured (use sections, headings, tables, and diagrams).
- Have a clear flow from:
  - definitions →
  - hierarchy/transformation →
  - constructed trees →
  - applications →
  - AI usage log.
- Be easy for another student to read and learn from.

Feel free to use AI to suggest a good outline, but you must decide and finalize the layout yourself.

f. AI Usage Log (Q&A Table)

Every time you use an AI copilot service for this assignment, record:

- Index (1, 2, 3, …)
- Prompt (what you asked)
- Service (e.g., ChatGPT, Gemini, Copilot, …)

Example log table:

Index	Prompt	Service
1	Assist me to have the definition of general tree, binary tree, complete binary tree, binary search tree, AVL tree, red-black tree, max heap and min heap for self-learning.	ChatGPT
2	Explain the difference between AVL tree and Red-Black tree in terms of balancing strategy and use cases.	Gemini
…	…	…

Place this table at the end of your report.

4. Evaluation (100 pts)

A possible breakdown (you can adjust if needed):

a. Concept definitions (20 pts)

- Correctness and clarity of all 8 tree type definitions.

b. Hierarchy & transformation explanation (20 pts)

- Clear diagram / explanation of how each tree variant evolves from the general tree.
- Correct identification of constraints/invariants.

c. Tree constructions & visualizations (25 pts)

- Correct constructions for each tree type using the given integers.
- Proper screenshots and tool URLs.

- Consistent insertion / heap-building strategy descriptions.

d. Applications & explanations (20 pts)
   - One application per tree type.
   - Clear explanation linking data structure properties to the application.

e. Report organization & AI usage log (15 pts)
   - Logical report structure and readability.
   - AI log completeness (all prompts listed with service names).
   - Thoughtful use of AI as a learning assistant, not as a copy-paste generator.

Student ID:1113715 Student Name:曾喜杰

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Student Worksheet Companion

Due date: 2025.12.30 23:59:59

Academic Integrity and AI Usage Statement

In this assignment, you must use AI tools (such as ChatGPT, Gemini, Claude, Grok, M365 Copilot, etc.) as learning assistants, but you must also take full responsibility for understanding and organizing your own work.

1. Permitted Use of AI Tools

   You may use AI to:

   - Review or clarify definitions and concepts.
   - Compare different tree data structures.
   - Get suggestions for report layout or examples.
   - Ask for explanations of algorithms (e.g., BST insertion, AVL rotation, heapify process).

   You should read, think about, and rewrite the content in your own words.

2. Not Permitted
   - Do not copy/paste AI-generated content directly as your final answer.
   - Do not ask AI to draw the final diagrams or directly produce the final tree screenshots.
   - Do not ask AI to complete the whole assignment report for you.
3. Your Responsibility
   - You are responsible for understanding the definitions and algorithms.
   - You are responsible for verifying whether AI answers are correct or not.
   - You must produce your own original explanations and diagrams.
4. AI Usage Log
   - You must record all AI queries related to this assignment.
   - At the end of your report, include an AI Usage Log table with: Index, Prompt, AI service name.

   By submitting this assignment, you acknowledge that you have used AI tools only as study aids, and that the final content of this assignment represents your own understanding and work.

Section 1. Definitions of Tree Variants

Task: Write your own definitions for each tree type. You may use AI for learning, but rewrite in your own words.

1. General Tree

   Definition:分層式的資料結構，由 root 與 node 組成，由 root 開始向下往其他 node 連

接，連接的線稱為 edge，相互直接連接且在上的 node 稱為 parent;在下的稱為 child，每個 node 能有任意的 children，children 的數量稱為 degree，從 root 開始算每往下一層 edge 為一個 level，tree 中最大的 level 稱為 height，在同一 level 且有同個 parent 的 node 互為 sibling。沒有任何 children 的 node 稱為 leaf node。

2. Binary Tree
   Definition:一種特殊的 tree，每個 node 最多只能有兩個 children，分別為 left child 與 right child，其餘與 tree 相同。

3. Complete Binary Tree
   Definition:一種 binary tree，除最後一層外，其餘 node 必須是滿的，且最後一層 node 必須靠左排列。

4. Binary Search Tree (BST)
   Definition:一種 binary tree，對於所有 node 其 left child 必須小於該節點，right child 必須大於該節點

5. AVL Tree
   Definition:一種嚴格平衡的 binary search tree，任何 node 的 left&right child 高度差不能超過 1，如果操作過後(如 insert、delete)造成 tree 失衡，則會透過 rotation 修正。

6. Red-Black Tree
   Definition: 一種弱平衡的二元搜尋樹。透過將 node 標記為紅色或黑色，並遵守特定規則(如：根是黑的、紅接黑、路徑上黑節點數相同)來確保樹的高度大致平衡，雖然不像 AVL 那麼嚴格，但在插入刪除時效率較高。

7. Max Heap
   Definition: 一種完全二元樹 (Complete Binary Tree)，且滿足堆積性質：parent 的值永遠大於或等於 children 的值 (Root 是最大值)。

8. Min Heap
   Definition: 一種完全二元樹 (Complete Binary Tree)，且滿足堆積性質：parent 的值永遠小於或等於 children 的值 (Root 是最小值)。

Section 2. Tree Family Hierarchy and Transformations

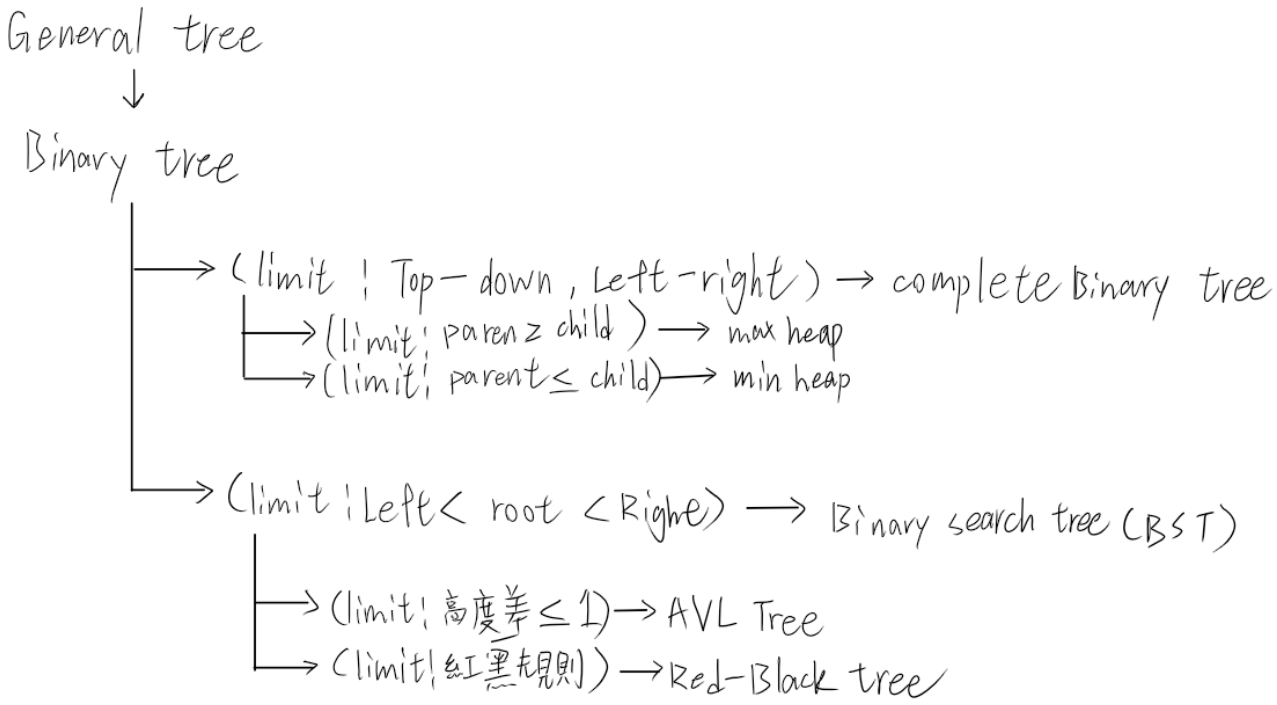
Task: Show how these structures are related (general → specialized). Use a simple diagram and explanations of what constraints are added at each step.

2.1 Tree Family Diagram

You may draw this by hand and paste a photo, or use drawing tools.

Student ID:1113715 Student Name:曾喜杰

Suggested chain example (you may extend or adjust):
General Tree → Binary Tree → Complete Binary Tree
Binary Tree → Binary Search Tree → AVL / Red-Black
Binary Tree → Max Heap / Min Heap
Your Diagram:



2.2 Explanation of Transformations
Fill in what new property or constraint is added at each step.

From	To	New property / constraint added
General Tree	Binary Tree	Max degree = 2
Binary Tree	Complete Binary Tree	新增結構限制：必須從上到下、由左至右填滿節點，不能有空洞 (除了最後一層右側)。
Binary Tree	Binary Search Tree	新增順序限制：左子節點 < 父節點 < 右子節點
BST	AVL Tree	新增平衡限制：任意節點的左右子樹高度差絕對值 ≤ 1。
BST	Red-Black Tree	新增著色規則：利用紅黑顏色規則來確保最長路徑不超過最短路徑的兩倍。
Binary Tree	Max Heap	新增堆積性質：父節點必須 ≥ 子節點。
Binary Tree	Min Heap	新增堆積性質：父節點必須 ≤ 子節點。

Section 3. Tree Constructions Using Given Integers
Given integers (fixed for all parts):
37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

Student ID:1113715 Student Name:曾喜杰

Task: For each tree type below, construct the tree using these integers, take a screenshot of the tree from your chosen tool, record the tool name/URL, and describe the insertion / heap-building procedure.

3.1 Binary Tree

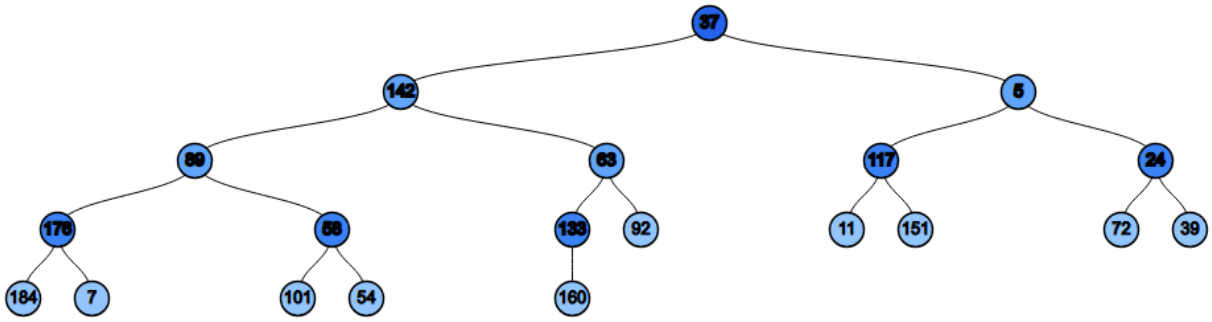
Tool name / URL:

https://treeconverter.com/?input=37,+142,+5,+89,+63,+117,+24,+176,+58,+133,+92,+11,+151,+72,+39,+184,+7,+101,+54,+160

Construction / insertion description:

Insert elements in level-order sequence (top-to-bottom, left-to-right) at the first available position.

Screenshot of Binary Tree (paste below):



3.2 Complete Binary Tree

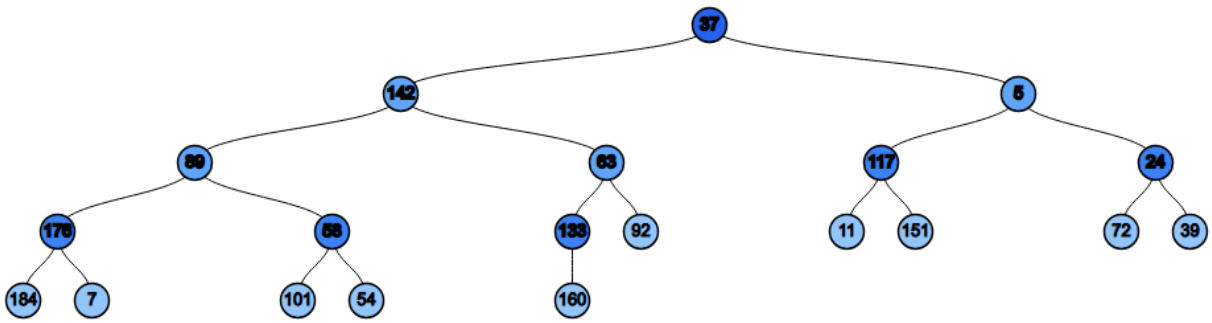
Tool name / URL:

https://treeconverter.com/?input=37,+142,+5,+89,+63,+117,+24,+176,+58,+133,+92,+11,+151,+72,+39,+184,+7,+101,+54,+160

Construction / insertion description:

Construct by filling nodes level by level, from left to right, ensuring no gaps in the array representation.

Screenshot of Complete Binary Tree (paste below):



3.3 Binary Search Tree (BST)

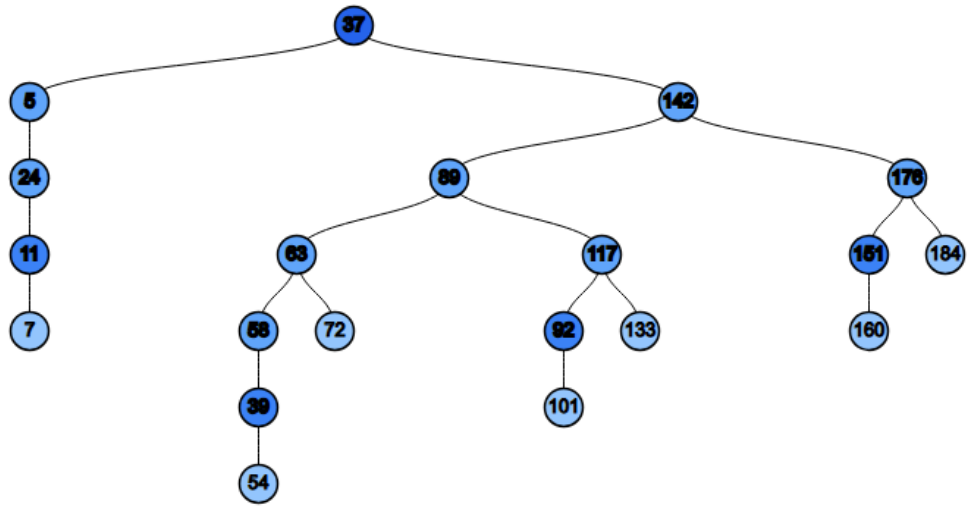
Tool name / URL:

https://treeconverter.com/?input=37,+142,+5,+89,+63,+117,+24,+176,+58,+133,+92,+11,+151,+72,+39,+184,+7,+101,+54,+160

Insertion rule (e.g., “insert in given order using BST rules”):

Insert each integer sequentially. For every node, values smaller than the node go to the left subtree, and values larger go to the right subtree.

Screenshot of BST (paste below):



3.4 AVL Tree

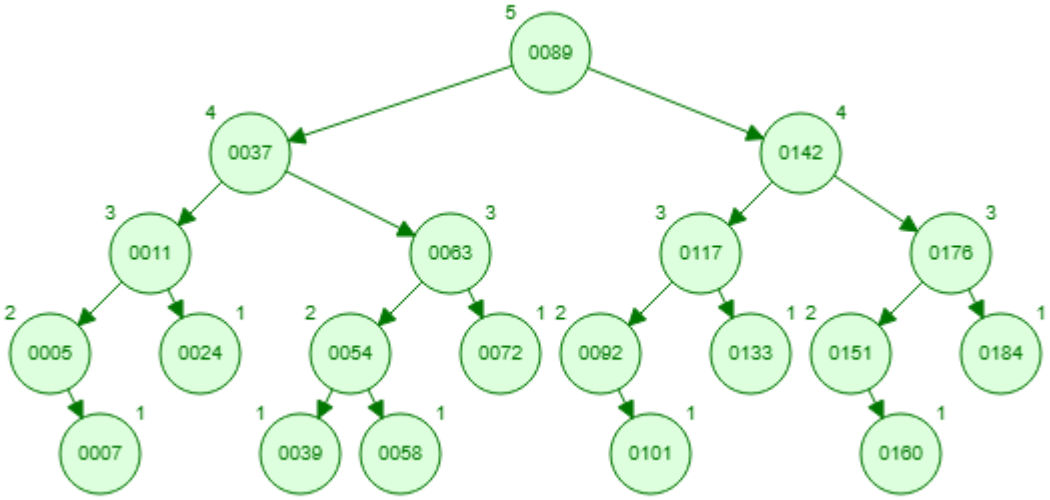
Tool name / URL:

https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

Insertion & balancing description:

Insert as in a BST, but perform rotations (LL, RR, LR, RL) automatically whenever the balance factor of a node becomes 2 or -2 to maintain strict height balance."

Screenshot of AVL Tree (paste below):



3.5 Red-Black Tree

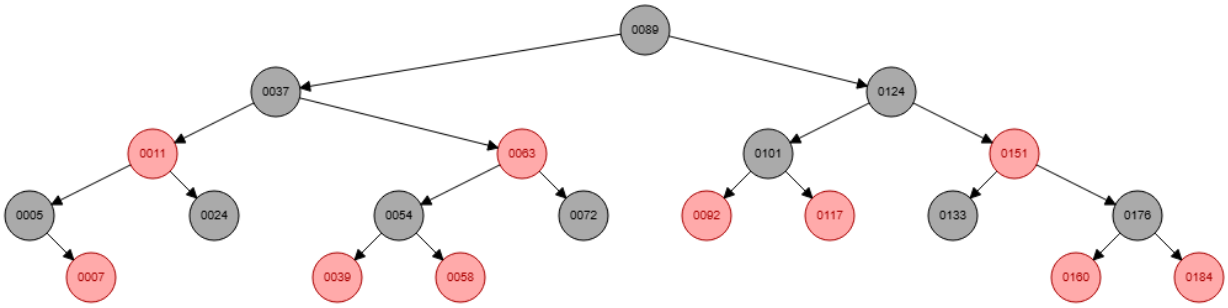
Tool name / URL:

Insertion & balancing description:

Insert as in a BST. New nodes are red. Use recoloring and rotations to ensure no two red nodes are adjacent and black-height is consistent.

https://www.cs.usfca.edu/~galles/visualization/RedBlack.html

Screenshot of Red-Black Tree (paste below):



3.6 Max Heap

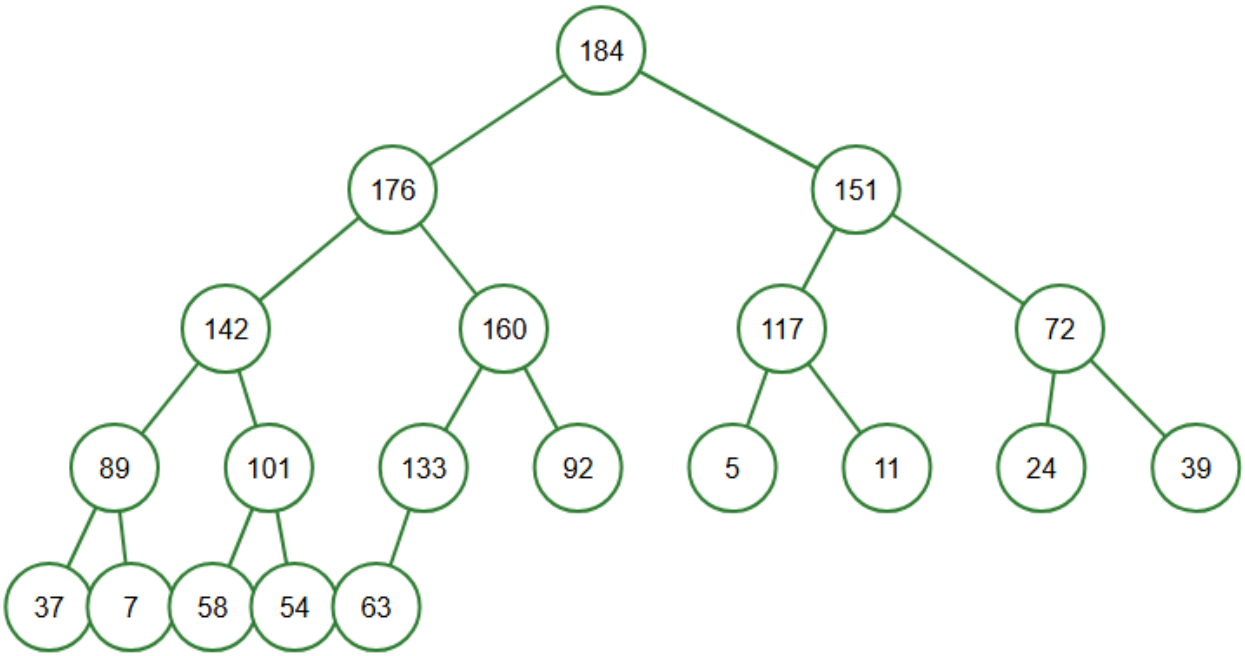
Tool name / URL:

https://sercankulcu.github.io/files/data_structures/slides/Bolum_08_Heap.html

Construction / heap-building description (e.g. heapify, insert-and-sift-up):

Insert elements one by one at the end (bottom-left available spot) and perform 'Sift Up' (Bubble Up) to restore the max-heap property (Parent ≥ Child).

Screenshot of Max Heap (paste below):



3.7 Min Heap

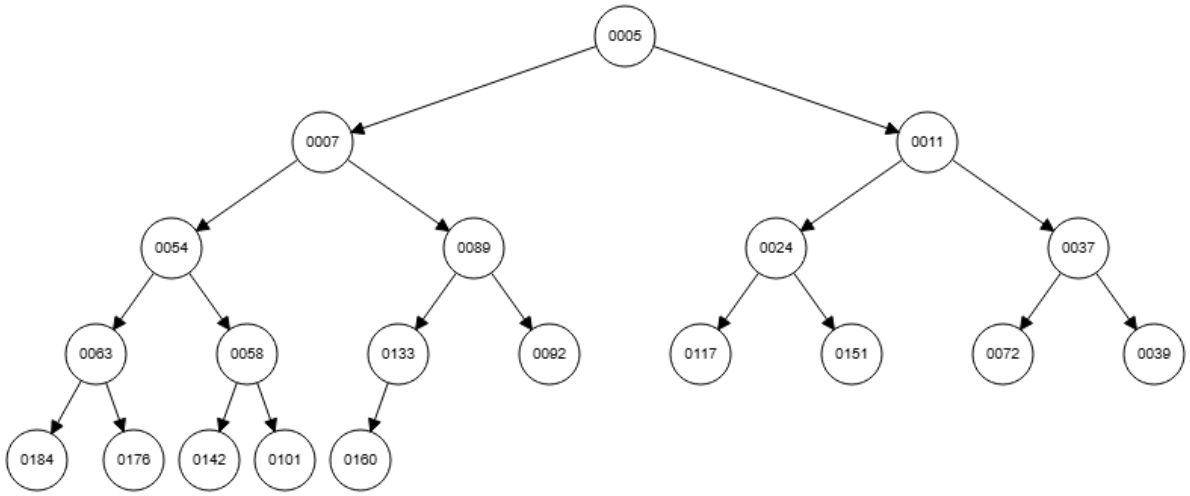
Tool name / URL:

https://algoanim.ide.sk/index.php?page=showanim&id=51

Construction / heap-building description:

Insert elements one by one and perform 'Sift Up' to restore the min-heap property (Parent ≤ Child)

Screenshot of Min Heap (paste below):



Section 4. Application Examples

Task: For each tree type, choose one application and explain why this tree is suitable.

Tree Type	Application Example (name / context)	Why this tree fits (properties that matter)
Binary Tree	Expression Trees	二元樹的階層結構非常適合表示數學運算式（例如 a + b * c）。葉節點代表運算元 (operands)，內部節點代表運算子 (operators)，樹的層級自然反映了運算的優先順序，讓電腦能正確計算複雜公式。
Complete Binary Tree	Binary Heaps	完全二元樹的節點是由上而下、由左至右緊密排列，沒有空洞。這使得我們可以直接使用陣列 (Array) 來儲存整棵樹，不需要額外儲存左右子節點的指標 (pointers)，大幅節省記憶體空間並提升存取效率。
Binary Search Tree	Simple Dictionary or Symbol Table	BST 具有「左小右大」的排序性質，這讓我們可以使用類似二分搜尋法的概念來尋找資料。在平均情況下，搜尋、插入與刪除的時間複雜度皆為 O(log n)，比傳統線性搜尋更有效率。
AVL Tree	Lookup-intensive Databases	AVL 樹保證絕對的高度平衡（左右子樹高度差不超過 1）。對於「搜尋次數遠多於寫入次數」的系統來說，雖然插入時的旋轉成本較高，但它能保證搜尋永遠

		維持在最佳效率 O(log n)，不會像一般 BST 那樣退化成串列。
Red-Black Tree	C++ STL map or Java TreeMap	紅黑樹是一種「弱平衡」的二元搜尋樹。相比於 AVL 樹，它在頻繁插入和刪除時需要的旋轉重整次數較少，能夠在「搜尋效率」與「修改效率」之間取得很好的平衡，適合用於通用的資料儲存容器。
Max Heap	Priority Queue	Max Heap 的特性保證「根節點永遠是最大值」。這讓系統可以在 O(1) 的時間內快速抓出目前優先權最高的工作來執行，而不需要掃描整個佇列，非常適合需要即時處理最高優先級任務的場景。
Min Heap	Dijkstra's Algorithm	在計算地圖最短路徑時，演算法需要不斷取出「目前距離起點最近」的節點來進行擴展。Min Heap 保證根節點永遠是最小值，讓演算法能以最高效率取得當前最短邊，加速路徑計算過程。

Section 5. Reflection on Tree Family and Performance (Optional but recommended)

Among BST, AVL, and Red-Black trees, which one would you pick for:

Mostly search (few updates)? Why?

AVL Tree。因為 AVL 樹維持著比紅黑樹更嚴格的高度平衡（Strictly Balanced），這使得它的樹高在相同節點數下通常比紅黑樹更低。較低的樹高意味著在進行搜尋時需要遍歷的路徑更短，因此在「讀多寫少」的情境下，AVL 樹能提供最快的查詢速度。

Frequent insertions and deletions? Why?

Red-Black Tree。因為紅黑樹是「弱平衡」的，它允許樹的高度稍微不平衡，這使得在進行插入或刪除操作時，觸發旋轉（Rotation）和重新著色（Recoloring）的頻率比 AVL 樹低。對於需要頻繁修改資料的應用，紅黑樹的整體綜合效能會優於 AVL 樹。

If you must store these 20 integers for static search only (no updates), which structure or representation would you prefer (sorted array + binary search, BST, AVL, etc.)? Why?

因為資料是靜態的（不會再新增或刪除），我們不需要樹狀結構帶來的動態修改彈性。使用陣列可以省下儲存指標（Pointers）的記憶體空間，且陣列在記憶體中是連續的，具有較好的快取局部性（Cache Locality），查詢效率極佳。

Student ID:1113715 Student Name:曾喜杰

Section 6. AI Usage Log (Required)

Task: Record every time you ask an AI assistant about this assignment.

Index	Date / Time	AI Service (ChatGPT, Gemini, etc.)	Your Full Prompt / Question
1	2025/12/30 19:30	Gemini	Assist me to define General Tree, Binary Tree, Complete Binary Tree, BST, AVL, Red-Black Tree, Max Heap, and Min Heap for my assignment.
2	2025/12/30 19:55	Gemini	Explain the hierarchy and transformation constraints from General Tree to specialized trees like AVL and Heaps.
3	2025/12/30 20:30	Gemini	How to construct the specific trees using the integer list: 37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160.
4	2025/12/30 21:00	Gemini	Provide real-world application examples for Binary Tree, Complete Binary Tree, BST, AVL, Red-Black Tree, Max Heap, and Min Heap, and explain why they fit.

You may extend this table as needed.